

Fine-Tuning a Code LLM to Generate Interactive Geometry-Diagram Constructions

Dataset Construction, Method, and Results

Aditya Dhull

June 2026

Abstract

We fine-tune **Qwen2.5-Coder-14B-Instruct** to translate a natural-language Olympiad geometry problem into an *ordered GeoGebra construction* (a JSON list of steps) that renders an interactive, draggable diagram. Training uses **QLoRA** (4-bit base + low-rank adapters) on a **292-problem** corpus that was assembled and verified by a custom headless-GeoGebra pipeline: every construction renders, every stated claim is numerically checked, every entity is consistently labelled **final/construction**, and no dead steps remain. On **39 held-out, unseen** problems the model produces **100%** parseable JSON and **85%** render-valid figures, a ~ 14 -point improvement over an earlier 14B run on the un-cleaned data (71%).

1 Problem and Task

The system maps a problem statement (e.g. “Let ABC be a triangle with incentre I ; the incircle touches BC at $D...$ ”) to a JSON object `{"construction_steps": [...]}`. Each step has a **name**, a **type** (`final` = shown, `construction` = hidden helper), a **class** (`point/line/segment/circle/...`), an optional **label**, and a **command** in native GeoGebra syntax that references earlier steps. Steps must be in constructible order; free base points use coordinates and everything else is derived (`Midpoint`, `Intersect`, `Circle`, `AngleBisector`, `Reflect`, ...). Because the output is essentially *code* with strict syntax and ordering constraints, the task is well suited to a code-specialised LLM.

2 Dataset Construction and Verification

The corpus has three sources, all passed through the same verification gate.

(1) Gold seed — 82 problems. Hand-curated, hand-checked problems that define the schema and quality bar.

(2) Generated — 150 problems. Authored in-session and gated by a headless-GeoGebra validator plus machine-checkable semantic assertions (`collinear`, `concylic`, `concurrent`, `tangent`, `Ptolemy`, ...). They span classic configurations: Miquel/Euler-line/nine-point/Feuerbach, Simson/Steiner, Pascal/Pappus/Desargues/Monge, Ptolemy, power-of-a-point/Menelaus/Ceva, symmedian/Lemoine, excentral/Nagel/Gergonne, Napoleon/Fermat, and quadrilateral metric theorems.

(3) New — 60 problems. Generated in two rounds. Round 1 used agents anchored on classic lemmas; a strict adversarial audit (below) kept only $\sim 25\%$, because most duplicated the existing 150 or carried logic defects. Round 2 used four *novel-territory* agents (special triangle centres; circle/radical/inversion/pole-polar; concrete metric problems with numeric answers; quadrilateral lemmas), each *numerically self-verifying every claim* before returning it — yielding a far higher survival rate.

Verification gate. Every candidate is checked for:

1. **Renderability** — each step builds a real object across multiple random-parameter renders in a real (headless) GeoGebra engine.
2. **Claim fidelity** — full-precision coordinates are probed and the statement’s assertion is tested numerically (collinearity, concyclicity, equal lengths, perpendicularity, ratios, concurrency, or an exact value).
3. **Meaningful steps** — no step is built and then left unused.
4. **Visibility** — every object named in the statement is **final**; pure scaffolding is **construction**.
5. **Novelty** — cross-dataset near-duplicate scan against all other problems.

Source	Problems	In training split
Gold (hand-curated)	82	71
Generated (verified)	150	130
New (audited, 2 rounds)	60	52
Total	292	253

Table 1: Final corpus. The held-out evaluation set is the remaining 39 problems (11 gold + 20 generated + 8 new), never seen in training. The 253 training problems are paraphrase-augmented to **850** examples.

3 Base Model Selection

We chose **Qwen2.5-Coder-14B-Instruct** (Apache-2.0). Rationale:

- **Code specialisation.** The target is GeoGebra command syntax; a top open *code* model minimises command hallucination.
- **Size fit.** In 4-bit it is ~ 9.3 GB and fits a single 16 GB GPU (Kaggle T4/P100) with room for activations and adapters.
- **Right capacity for the data.** A 32B model was considered but rejected: it does not fit one 16 GB GPU (needing fragile multi-GPU model parallelism) and is over-sized for a 292-problem LoRA task. 14B is the sweet spot.
- **Licensing & cost.** Apache-2.0, trainable for free on Kaggle.

4 Fine-Tuning Method

4.1 QLoRA and why 4-bit

We use **QLoRA**: the pretrained base is loaded in **4-bit** (NF4, double quantisation) and *frozen*; only small **low-rank adapter** matrices are trained (in higher precision). 4-bit quantisation is the enabling choice:

Precision	Base weights (14B)	Fits one 16 GB T4?
fp16 / bf16 (16-bit)	~28 GB	No (out-of-memory)
int8 (8-bit)	~14 GB	Tight, little headroom
nf4 (4-bit)	~ 9.3 GB	Yes, with room

Table 2: Why 4-bit: it shrinks the frozen base enough to leave VRAM for activations and optimiser state, making single-GPU fine-tuning of a 14B model feasible on free hardware. Quantisation error is absorbed by the trainable adapters, so accuracy loss is negligible for this task.

Only **137,625,600** parameters are trained (**0.92%** of 14.9B), so the saved artefact is a ~550 MB adapter rather than a full checkpoint.

4.2 Configuration

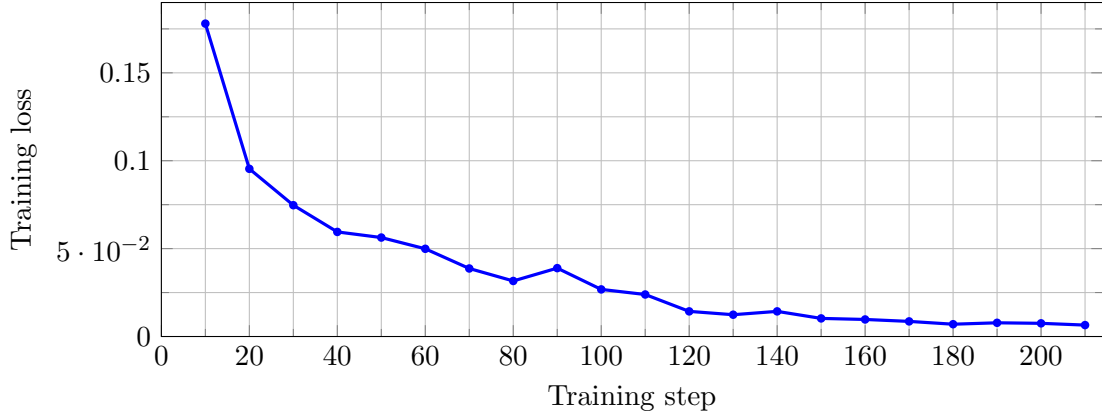
Training used the Unsloth library (single GPU, paged 8-bit AdamW), with the loss computed *only on the assistant JSON* (prompt tokens masked).

Setting	Value
Base model	Qwen2.5-Coder-14B-Instruct (4-bit NF4)
Adapter (LoRA)	$r = 32$, $\alpha = 32$, dropout 0
Target modules	q,k,v,o,gate,up,down_proj
Trainable params	137.6M (0.92%)
Batch / grad-accum	1×8 (effective batch 8)
Epochs / steps	2 / 214
Learning rate	2×10^{-4} , cosine, 5% warmup
Max sequence length	1536
Optimiser	adamw_8bit ; loss on responses only
Hardware	1× Kaggle GPU (T4/P100, 16 GB)
Wall-clock	9871 s $\approx$ 2 h 44 m

Table 3: Fine-tuning configuration.

4.3 Training dynamics

The loss decreased smoothly and monotonically with no instability.



Final training loss ≈ 0.006 . Such a low value is expected when learning exact construction syntax from paraphrase-augmented data; the honest quality signal is therefore the held-out render rate (§6).

5 Inference and Post-Processing

At inference the model is loaded as base+adapter (4-bit) and decoded greedily. Three light, deterministic fixes are applied to the raw output:

1. lower-case the `class` field;
2. reorder `AngleBisector(P,Q,R)` so the named vertex is the middle argument;
3. the interactive renderer auto-corrects a collapsed `Intersect` index (when a new point lands on an existing one).

6 Results on 39 Held-Out Problems

The model was evaluated on the 39 never-seen problems by generating a construction for each and re-rendering it in headless GeoGebra. A prediction counts as *render-valid* only if every step builds across multiple random renders; an unparsed prediction counts as a failure.

Metric	This run (14B, cleaned 292-set)	Prior 14B (un-cleaned data)
Parseable JSON	39/39 = 100%	100%
Render-valid	33/39 = 85%	71%

Table 4: Held-out results. The cleaned, audited, visibility-labelled corpus lifts render-validity by ~ 14 points.

Failure analysis. All six failures are isolated geometry slips on the hardest constructions — none are JSON or formatting errors.

7 Conclusion and Future Work

A 14B code LLM, QLoRA-tuned on a small but rigorously verified 292-problem corpus, generalises to unseen Olympiad geometry problems at **85%** render-validity. The decisive factors were (i) a

#	Cause	Category
1	<code>Angle(A,B,C)</code> used where <code>AngleBisector</code> was needed	wrong command
2	<code>Line(A,Xdoubleprime)</code> — <code>Xdoubleprime</code> never defined	undefined reference
3	cevian feet <code>Ap/Bp/Cp</code> not built / degenerate	undefined / degenerate
4	two lines parallel for the chosen coords $\Rightarrow$ no intersection	degenerate config
5	<code>Intersect(...,2)</code> — second intersection does not exist	invalid index
6	long incircle/parallel chain with a bad intermediate	complex hallucination

Table 5: The six render failures. Several (wrong command, invalid index) are the class a `generate` $\rightarrow$ `validate` $\rightarrow$ `repair` loop would recover, so usable quality is slightly above the raw 85%.

verification pipeline that guarantees every training target renders, matches its stated claim, and is consistently labelled, and (ii) 4-bit QLoRA, which makes single-GPU fine-tuning of a 14B model feasible on free hardware. The clearest next lever is a *generate* $\rightarrow$ *validate* $\rightarrow$ *repair* loop that feeds a failing step and its GeoGebra error back to the model for one or two retries, which would recover the command/index failures and push render-validity past 90%.